

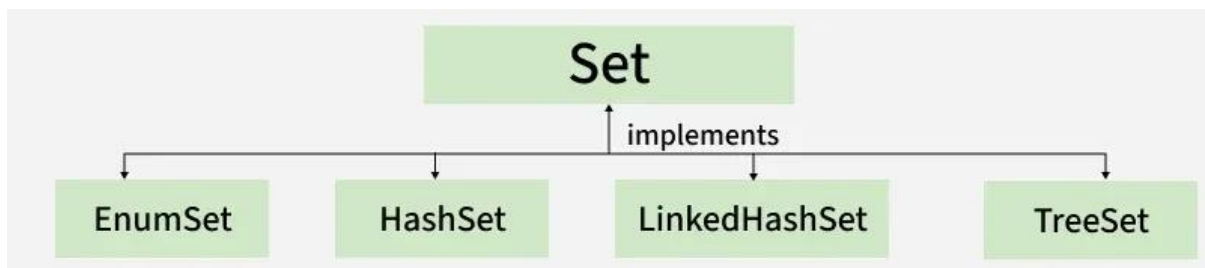
Java Collection Framework

Set Interface:-

- The Set interface is part of the Java Collection Framework.
- It is present in the java.util package.
- It stores unique elements and does not allow duplicates.
- The Set interface does not maintain any specific order.

Hierarchy of Java Set interface :-

The image below demonstrates the hierarchy of Java Set interface.



- **Set** - Extends Collection, stores unique elements.
- **EnumSet** - A specialized Set for use with enum types, very fast and memory-efficient.
- **HashSet** - Implements Set using hashing (unordered).
- **LinkedHashSet** - Extends HashSet, maintains insertion order.
- **TreeSet** - Implements NavigableSet, stores elements in ascending order.

Methods of Set Interface :-

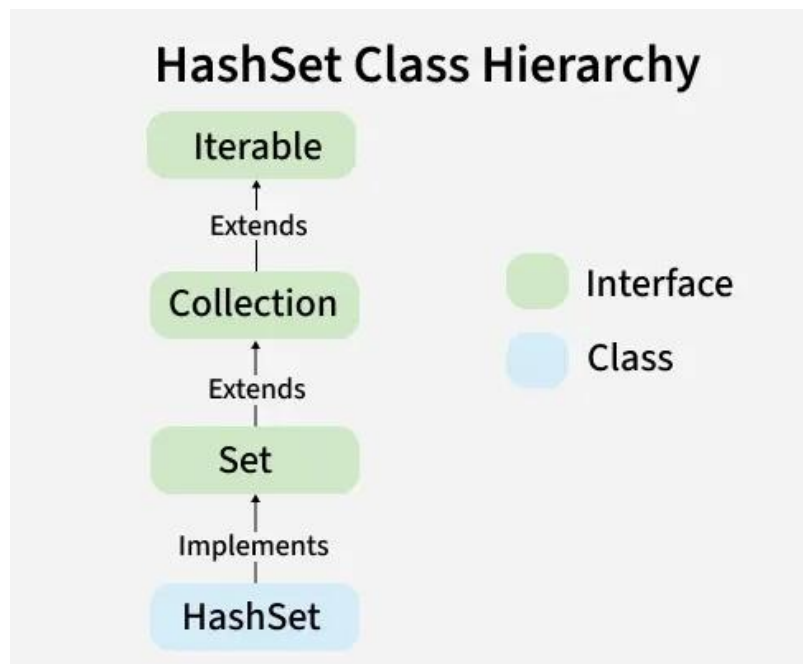
Method	Description
add(element)	Adds element if not already present. Returns true if added.
addAll(collection)	Adds all elements from the given collection.
clear()	Removes all elements from the set.
contains(element)	Checks if the set contains the specified element.

Method	Description
containsAll(collection)	Checks if the set contains all elements from the given collection.
hashCode()	Returns the hash code of the set.
isEmpty()	This method is used to check whether the set is empty or not.
iterator()	This method is used to return the iterator of the set.
remove(element)	Removes the specified element from the set.
removeAll(collection)	Removes all elements in the given collection from the set.
retainAll(collection)	Retains only elements present in the given collection.
size()	Returns the number of elements in the set.
toArray()	This method is used to form an array of the same elements as that of the Set.

Classes that implement the Set interface :-

1) HashSet :-

- HashSet is a class in Java that implements the Set interface.
- It stores unique elements and does not allow duplicates.
- It does not maintain insertion order of elements.
- It allows one null value and provides fast access using hashing.
- It is used to store unordered and unique data efficiently



Syntax :-

`HashSet<data_type> set_name = new HashSet<data_type>();`

Methods of HashSet :-

Method	Description
<code>add(E e)</code>	Used to add the specified element if it is not present, if it is present then return false.
<code>clear()</code>	Used to remove all the elements from the set.
<code>contains(Object o)</code>	Used to return true if an element is present in a set.
<code>remove(Object o)</code>	Used to remove the element if it is present in set.

Method	Description
iterator()	Used to return an iterator over the element in the set.
isEmpty()	Used to check whether the set is empty or not. Returns true for empty and false for a non-empty condition for set.
size()	Used to return the size of the set.
clone()	Used to create a shallow copy of the set.

Example :-

```
package collections;

import java.util.HashSet;

public class hashset {

    public static void main(String[] args) {
        //using HashSet non-generic collection
        HashSet hs=new HashSet();

        hs.add("Sagar"); //1.add method
        hs.add('A');
        hs.add(23);
        hs.add(78.9);
        hs.add(23);

        System.out.println("Non-Generic HashSet is: "+hs);

        System.out.println("contains: "+hs.contains("Rahul")); //2.contains method

        hs.remove(23); //3.remove method
        System.out.println("After removing object hashset is: "+hs);

        System.out.println("is empty: "+hs.isEmpty()); //4.isempty method

        hs.clear(); //5.clear method
        System.out.println("After clear method hashset is: "+hs);

        //using HashSet generic collection
        HashSet<Integer>hs1=new HashSet<Integer>();
        hs1.add(10);
        hs1.add(20);
        hs1.add(30);

        System.out.println("Generic HashSet is: "+hs1);
```

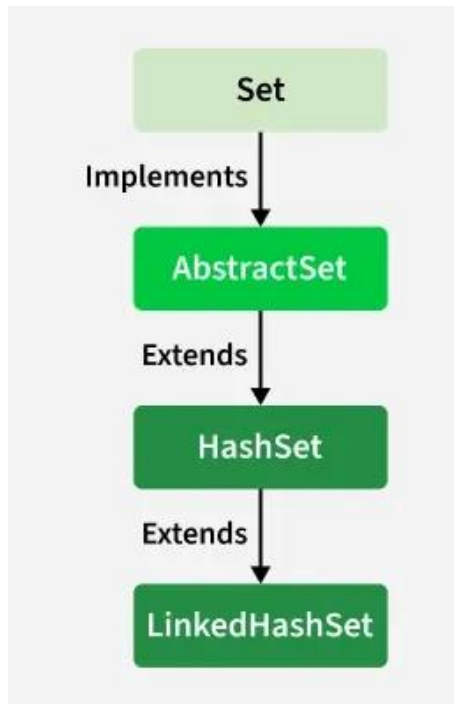
```
        for(Integer i:hs1)
        {
            System.out.println(i);
        }
    }
}
```

Output :-

```
Non-Generic HashSet is: [A, Sagar, 23, 78.9]
contains: false
After removing object hashset is: [A, Sagar, 78.9]
is empty: false
After clear method hashset is: []
Generic HashSet is: [20, 10, 30]
20
10
30
```

LinkedHashSet :-

- LinkedHashSet implements the Set interface and is part of java.util package.
- It stores unique elements and maintains insertion order.
- It uses a hash table with a linked list internally.
- It allows one null value and provides fast access.



Syntax :-

`LinkedHashSet<data_type> set_name = new LinkedHashSet<data_type>();`

Methods of LinkesHashSet :-

Method	Description
<code>add(E e)</code>	Adds an element if it's not already present.
<code>addAll(Collection c)</code>	Adds all elements from the specified collection.
<code>clear()</code>	Removes all elements from the set.
<code>contains(Object o)</code>	Checks if the set contains the specified element.

Method	Description
containsAll(Collection c)	Checks if the set contains all elements from the given collection.
remove(Object o)	Removes the specified element from the set.
removeAll(Collection c)	Removes all matching elements from the set.
retainAll(Collection c)	Keeps only elements present in the given collection.
isEmpty()	Checks if the set is empty.
size()	Returns the number of elements in the set.
iterator()	Returns an iterator over the elements.
toArray()	Returns an array containing all elements.
toArray(T[] a)	Returns an array of the specified type containing all elements.
hashCode()	Returns hash code of the set.
equals(Object o)	Compares this set with another set.
clone()	Creates a shallow copy of the set.
toString()	Returns a string representation of the set.
splititerator()	Returns a Spliterator for this set.

Example :-

```
package collections;

import java.util.HashSet;
import java.util.LinkedHashSet;

public class linkedhashset {

    public static void main(String[] args) {
        //using LinkedHashSet non-generic collection
        LinkedHashSet lhs=new LinkedHashSet();
        lhs.add("dscd");
        lhs.add(20);
        lhs.add(65.3);
        lhs.add(65.3);

        System.out.println("Non-Generic LinkedHashSet is: "+lhs);

        lhs.remove(20); //remove method
        System.out.println("After removing object LinkedHashSet is: "+lhs);

        //using LinkedHashSet generic collection
        LinkedHashSet<Integer>lsh1=new LinkedHashSet<Integer>();
        lsh1.add(10);
        lsh1.add(20);
        lsh1.add(20);
        lsh1.add(30);

        System.out.println("Generic LinkedHashSet is: "+lsh1);

        for(Integer i:lsh1)
        {
            System.out.println(i);
        }
    }
}
```

Output :-

```
Non-Generic LinkedHashSet is: [dscd, 20, 65.3]
After removing object LinkedHashSet is: [dscd, 65.3]
Generic LinkedHashSet is: [10, 20, 30]
10
20
30
```

Difference Between List Interface & Set Interface :-

List	Set
1. The List is an indexed sequence.	1. The Set is an non-indexed sequence.
2. List allows duplicate elements	2. Set doesn't allow duplicate elements.
3. Elements by their position can be accessed.	3. Position access to elements is not allowed.
4. Multiple null elements can be stored.	4. Null element can store only once.
5. List implementations are ArrayList, LinkedList, Vector, Stack	5. Set implementations are HashSet, LinkedHashSet.